Strategic Master Plan: The NeuroSim Autonomous Racing Research Initiative

1. Executive Vision and Architectural Philosophy

This document articulates a comprehensive, fifteen-month research and engineering roadmap designed to construct **NeuroSim**, a high-fidelity, differentiable autonomous racing framework. Unlike standard engineering projects that prioritize immediate functionality, this plan adopts the rigor and long-term vision characteristic of premier AI research laboratories such as Google DeepMind. The objective is not merely to build a car that drives but to engineer a system that *understands* driving through the synthesis of model-based reinforcement learning (MBRL), differentiable physics, and probabilistic inference.
The "DeepMind-style" approach mandates that every engineering decision be grounded in theoretical first principles. We do not simply "train a model"; we formulate a hypothesis regarding latent dynamics and construct an architecture to test it. We do not simply "fix bugs"; we implement rigorous regression testing and ablation studies to ensure reproducibility. This master plan is structured to transform a capable engineer into a research-grade architect, systematically bridging the chasm between abstract mathematical theory and high-speed physical actuation.

1.1 The "Digital Twin First" Doctrine

The strategy explicitly decouples the development of the intelligence kernel from the physical constraints of the Brunel Racing platform in the initial phases. By prioritizing a "Digital Twin"—a high-fidelity, differentiable simulation environment—we enable rapid iteration cycles that are impossible in the physical world. This aligns with the "Sim-to-Real" paradigm where an agent learns robust policies in a vast distribution of simulated environments (Domain Randomization) before ever seeing a real track. Integration with the physical vehicle is treated as a deployment target, not a development dependency.

1.2 The NeuroSim Core Stack

The technical foundation of this initiative is the JAX ecosystem (JAX, Flax/Haiku, Optax). JAX is chosen over PyTorch for its functional purity, which aligns with the mathematical formulation of physics and control theory, and its superior performance in compiling complex gradient calculations via XLA (Accelerated Linear Algebra). This choice signals a commitment to cutting-edge research engineering, positioning the project at the frontier of current AI development practices.

---

2. Phase 0: The Theoretical Substrate (Months 1-3)

Objective: To construct the mathematical and physical intuition necessary to innovate, rather than merely implement. This phase is non-negotiable; deep research engineering requires fluency in the language of gradients, probabilities, and energy landscapes.

2.1 Module 0.1: Advanced Linear Algebra & Optimization

Deep learning is fundamentally the application of linear algebra at scale. The curriculum here moves beyond matrix multiplication to the spectral properties that govern training stability and feature representation.

- **Eigenvalue Decomposition & Spectral Theory:** Understanding eigenvalues is critical for analyzing the stability of dynamical systems and the curvature of loss landscapes. You will study how the Hessian matrix's eigenspectrum predicts whether a neural network will converge or oscillate.
    - *Actionable Task:* Implement a visualizer in Python that plots the loss landscape of a simple 2-layer MLP and computes the Hessian's eigenvalues at various points during training to visualize saddle points vs. local minima.[1]
- **Singular Value Decomposition (SVD):** SVD is the bedrock of dimensionality reduction and data compression. In the context of World Models, it provides intuition for how Variational Autoencoders (VAEs) compress high-dimensional camera feeds into compact latent states.
    - *Actionable Task:* Write a "from scratch" image compression algorithm using SVD on F1 telemetry heatmaps, analyzing the trade-off between reconstruction error (MSE) and rank reduction.
- **Calculus of Variations:** To understand modern physics-informed AI, one must understand optimization not just of parameters, but of functions. This ties directly to the Euler-Lagrange equations.
    - *Research Insight:* Reinforcement learning can be framed as a variational inference problem where we optimize a distribution over trajectories. Understanding variational calculus allows for deeper insights into maximum entropy RL methods like Soft Actor-Critic (SAC).[2]

2.2 Module 0.2: Lagrangian & Hamiltonian Mechanics

Standard "black-box" neural networks struggle to learn conservation laws (energy, momentum). To build a superior world model, we will bake physics *into* the network architecture.

- **The Principle of Least Action:** Nature is lazy; systems evolve to minimize action. This principle is the theoretical bridge between classical mechanics and optimal control.
    - *Study Focus:* Derive the Euler-Lagrange equation $\frac{d}{dt}(\frac{\partial L}{\partial \dot{q}}) - \frac{\partial L}{\partial q} = 0$ for a simple pendulum.
- **Hamiltonian Dynamics:** While Lagrangian mechanics focuses on configuration space $(q, \dot{q})$, Hamiltonian mechanics operates in phase space (q, p), offering symplectic structures that preserve volume in phase space—crucial for stable long-term

simulation.[3]
    - *DeepMind Connection:* DeepMind researchers often use Hamiltonian Neural Networks (HNNs) to learn dynamics that strictly conserve energy, preventing the "vanishing/exploding energy" problem common in standard RNNs forecasting physical systems.[4]
- **Project Artifact: The Differentiable Double Pendulum:**
    - *Goal:* Build a simulation of a double pendulum where the equations of motion are solved using a neural network trained to minimize the Hamiltonian energy divergence, rather than just MSE of position.
    - *Tech Stack:* JAX (for autodiff), Runge-Kutta integrators. This serves as the "Hello World" for your Physics-AI integration.[4]

2.3 Module 0.3: Probabilistic Modeling & Information Theory

Autonomous racing is decision-making under uncertainty. We must quantify what the car *doesn't* know.

- **Bayesian Inference:** Moving from point estimates (what is the friction?) to distributions (what is the probability distribution of friction?).
- **Variational Inference (VI) & ELBO:** The core of the "World Model" is the VAE. You must master the Evidence Lower Bound (ELBO) derivation to understand how to train a model that balances reconstruction accuracy with latent space regularity.
    - *Actionable Task:* Derive the KL-divergence term for a Gaussian VAE by hand, then implement it in JAX.
- **Markov Decision Processes (MDPs) & POMDPs:** The formal framework of RL. The real world is a Partially Observable MDP (POMDP) because sensors are noisy and occlusion exists.
    - *Research Insight:* The "World Model" essentially attempts to convert the POMDP of the real world into an MDP of the latent state, making control tractable.[7]

2.4 Module 0.4: Modern LLM & Transformer Architecture

Even for control tasks, the Transformer architecture is becoming dominant (Decision Transformers).

- **Attention Mechanisms:** Understand $Softmax(\frac{QK^T}{\sqrt{d_k}})V$ not just as a formula, but as a content-addressable memory retrieval system.
- **Tokenization & Embeddings:** How continuous signals (telemetry) can be discretized into tokens for sequence modeling.
    - *Actionable Task:* Train a nano-GPT from scratch (following Karpathy's blueprint) but on a dataset of *racing telemetry strings* instead of Shakespeare. This builds intuition on sequence prediction dynamics.

---

3. Phase 1: The Research Engineering Stack (Months

4-5)

Objective: To establish a professional, scalable software environment. "Research code" is often pejorative, implying messy, untestable scripts. We will write *Research Engineering* code: modular, tested, and reproducible.

3.1 The JAX Ecosystem: Flax & Haiku

We will transition from the imperative style of PyTorch to the functional paradigm of JAX.

- **JAX Fundamentals:** Master the "trinity": jax.grad (autodiff), jax.jit (compilation), and jax.vmap (vectorization).
    - *Insight:* vmap changes the game for RL. Instead of gathering experience from one agent, we can parallelize 10,000 agents on a single GPU without complex multiprocessing code. This allows for massive throughput in simulation.[9]
- **Flax/Haiku:** These libraries manage state (neural network weights) in JAX. While DeepMind often uses Haiku, Flax is gaining broader traction. We will adopt **Flax** for its flexibility or **Haiku** if strict adherence to DeepMind legacy patterns is desired. The plan assumes Flax for broader community support, but the principles transfer.
    - *Project Artifact:* **"JAX-ResNet-CIFAR"**. Re-implement a ResNet classifier in JAX/Flax. The goal is not accuracy, but to implement a clean training loop using TrainState, optax for optimization, and orbax for checkpointing.[11]

3.2 MLOps & Reproducibility Infrastructure

A model that cannot be reproduced does not exist.

- **Configuration Management (Hydra):** We will use Hydra to manage complex experimental configurations (learning rates, network architectures, physics parameters). This allows sweeping over parameters without changing code.[13]
- **Experiment Tracking (Weights & Biases):** Every run must be logged. Loss curves, gradient norms, and video rollouts of the agent must be streamed to a central dashboard.
    - *Standard:* Never train "blind". If a run crashes, the logs should tell you why.[15]
- **Containerization (Docker):** The project must run inside a Docker container to ensure that CUDA versions and JAX library versions are pinned. This eliminates "it works on my machine" issues.
- **The "Mono-Repo" Structure:**

```
neurosim/
├── configs/ # Hydra yaml configs
├── data/ # Immutable datasets
├── docker/ # Dockerfiles
├── neurosim/ # Source code
│ ├── agents/ # RL agents (PPO, SAC, Dreamer)
│ ├── envs/ # F1TENTH gym wrappers
│ ├── models/ # JAX neural network definitions
```

```
│ └── utils/ # Math utilities
├── scripts/ # Training and evaluation scripts
├── tests/ # Pytest unit tests
└── README.md
```

- *Actionable Task:* Initialize the repository. Set up pre-commit hooks that enforce black (formatting), isort (imports), and mypy (type checking). Pass a "Hello World" test in CI/CD (GitHub Actions).[16]

---

4. Phase 2: The Core — World Models & Model-Based RL (Months 6-8)

Objective: To build the cognitive engine. We move from model-free RL (reacting to the present) to model-based RL (predicting the future).

4.1 The Theory of World Models

We will implement the architecture popularized by the "Dreamer" series (DreamerV3).

- **Recurrent State Space Model (RSSM):** This is the heart of the system. It splits the state into a deterministic component (RNN) and a stochastic component (VAE).
  - *Deterministic Path:* $h_t = f(h_{t-1}, z_{t-1}, a_{t-1})$ (Memory)
  - *Stochastic Path:* $z_t \sim P(z_t|h_t)$ (Uncertainty)
  - *Insight:* The deterministic path allows long-horizon consistency, while the stochastic path handles the inherent unpredictability of the real world (e.g., tire slip, sensor noise).[8]

4.2 Building the Components in JAX

- **The Encoder (VAE):** Build a ConvNet that compresses LiDAR scans and telemetry into a latent vector.
  - *Challenge:* LiDAR data is sparse. You may need to process it as a 1D sequence or rasterize it into a 2D image.
- **The Dynamics Model (The "Dreamer"):** Train a model to predict the *next* latent state given the current state and action.
  - *Loss Function:* The objective is minimizing the KL divergence between the predicted prior (what we thought would happen) and the posterior (what actually happened, encoded from the next observation).
- **The Reward Predictor:** A small MLP that predicts the reward (progress along track) from the latent state.

4.3 Training in "The Dream"

Once the world model is trained, the agent (Actor-Critic) trains purely inside the model's "imagination".

- **The Actor:** Proposes actions to maximize predicted future value.
- **The Critic:** Estimates the value of latent states.
- *Project Artifact:* **"JAX-Dreamer-Lander"**. Before racing, verify your Dreamer implementation on a simple LunarLander or Pendulum environment. Prove that the agent can learn to fly without interacting with the real env, only the learned world model.[19]

---

5. Phase 3: The Simulation-to-Reality Gap (Months 9-11)

Objective: To prepare the mind for the body. A perfect simulation is a trap; we must engineer robustness against the imperfections of the real world.

5.1 F1TENTH Simulation Architecture

We will integrate the standard F1TENTH Gym (PyBullet-based) but wrap it in a JAX-compatible interface.

- **The Physics Engine:** The F1TENTH simulator uses a "Single Track Model" (bicycle model) with Pacejka tire dynamics.
    - *Deep Research Task:* Study the Pacejka Magic Formula. It describes the non-linear relationship between slip angle and tire force. This is the "limit of handling" where races are won or lost.[21]
- **Differentiable Wrapper:** While the base F1TENTH sim is not differentiable, we can build a *surrogate* differentiable model in JAX that approximates it. This allows us to use gradient-based planning (like Model Predictive Path Integral - MPPI) which is far more efficient than random shooting.

5.2 Domain Randomization (DR)

The "Sim-to-Real" gap exists because simulation parameters (friction, mass, motor inductance) never perfectly match reality.

- **Parameter Perturbation:** Instead of training on a friction coefficient of $\mu = 1.0$, we train on $\mu \sim U(0.8, 1.2)$.
- **Visual Randomization:** If using cameras, we randomize lighting, textures, and noise. For LiDAR, we inject Gaussian noise and "drop" random rays to simulate glass or absorption.
- **Latency Modeling:** Real robots have delay. We must simulate the 20-50ms delay between command generation and motor actuation. We do this by feeding the agent the observation from t but executing the action at $t + k$.[23]

5.3 System Identification (SysID)

Rather than just randomizing, we can try to *learn* the physical parameters.

- *Project Artifact:* **"Physics-ID Module"**. Train a small MLP that takes a sequence of (state, action, next_state) and outputs an estimate of the mass and friction coefficient. This estimate is fed into the World Model as a "context" vector, allowing the policy to adapt its driving style if it detects a slippery track.[25]

---

6. Phase 4: Autonomous Racing Agent (Months 12-14)

Objective: To synthesize the components into a racing stack. This moves beyond simply "staying on track" to "optimizing lap time".

6.1 The Planner: MPPI & MPC

Reinforcement Learning gives us a "reactive" policy. To be competitive, we need "deliberative" planning.

- **Model Predictive Control (MPC):** We use our JAX World Model to rollout thousands of potential trajectories parallelized on the GPU. We evaluate them using a cost function (maximize speed, minimize crash risk) and execute the first step of the best trajectory.
- **MPPI (Model Predictive Path Integral):** A sampling-based version of MPC that handles non-convex costs (like avoiding other cars) better than gradient-based MPC. JAX's vmap allows us to sample 10,000 trajectories in milliseconds.[26]

6.2 Learning the Racing Line

- **Reward Shaping:** We design a reward function that penalizes distance from the optimal racing line (calculated offline via convex optimization) but rewards speed.
- **Curriculum Learning:**
    1. *Level 1:* Drive slowly without crashing.
    2. *Level 2:* Follow the center line at speed.
    3. *Level 3:* Follow the optimal racing line.
    4. *Level 4:* Overtake static obstacles.
    5. *Level 5:* Overtake dynamic adversaries (other agents).[27]

6.3 The "Blind" Agent vs. The "Vision" Agent

We start with LiDAR-only (2D rays). Later, we incorporate camera data if compute permits.

- *Strategic Note:* LiDAR is robust and low-dimensional. For F1TENTH, it is the primary sensor. Do not get distracted by Computer Vision complexity unless the LiDAR policy is perfected.

---

7. Phase 5: Real-World Integration & Brunel Racing (Month 15+)

Objective: To cross the gap. This is where the rubber meets the road.

7.1 The Hardware Interface

We need a bridge between our JAX policy (Python) and the car's embedded system (typically ROS 2 on an NVIDIA Jetson).

- **ROS 2 Bridge:** Write a ROS 2 node that subscribes to /scan (LiDAR) and /odom (Odometry), converts them to a JAX array, queries the policy, and publishes to /drive.[22]
- **Latency Compensation:** Implement the latency modeling from Phase 3. The car moves while the calculation happens.

7.2 Safety Architecture

Research policies are stochastic and can hallucinate. We need a "Safety Filter" or "Guardian".

- **Automatic Emergency Braking (AEB):** A simple, non-AI rule-based system that overrides the AI if the Time-To-Collision (TTC) drops below a threshold. This runs at a high frequency (100Hz) on the car's MCU.[29]

7.3 Real-World Data Loop

- **Data Logging:** Every lap driven by the real car is recorded.
- **Fine-tuning:** This real data is fed back into the World Model to update the physics parameters (SysID). The simulation becomes more accurate, the agent retrains in the improved sim, and the cycle repeats. This is the **Data Flywheel**.[30]

---

8. The Software Development Lifecycle (SDLC) for Research

To build like DeepMind, you must treat your research code as a product. The "one-off script" mentality is the enemy of progress.

8.1 Repository Structure & Standards

- **Monorepo:** Keep simulation, modeling, and deployment code in one repository to ensure version consistency.
- **Typing:** Use Python type hints (typing.List, jax.numpy.ndarray) rigorously. This acts as self-documentation.
- **Config-Driven:** Never hardcode hyperparameters. Use config.yaml files via Hydra. This allows you to look back at a run from 3 months ago and know *exactly* what learning rate was used.[13]

8.2 Testing Strategy

- **Unit Tests:** Test individual mathematical functions. Does the calculate_reward() function return 0 for a crash? Does the VAE encoder output the correct shape?
- **Integration Tests:** Run a "dummy" training loop for 1 step to ensure the pipeline (Data

-> Model -> Loss -> Optimizer) is connected correctly.
- **Regression Tests:** Keep a "golden" model checkpoint. When you change the code, run an evaluation. If the performance drops by >5%, the test fails. This prevents "silent bugs" where the code runs but the math is broken.[31]

8.3 The "Research Log"

Maintain a research_log.md or a Notion/Obsidian database.

- *Entry Template:*
  - **Hypothesis:** "Adding Layer Norm to the GRU will stabilize gradients."
  - **Experiment:** Run ID #452 vs #453.
  - **Result:** "Gradients stable, but performance degraded."
  - **Conclusion:** "Layer Norm interferes with the recurrence; try Group Norm instead."
- This log is invaluable for writing papers and answering interview questions about *why* you made certain design choices.

---

9. Detailed Execution Timeline

The timeline is aggressive but modular. Sprints are 2 weeks long.

Phase 0: Foundations (Weeks 1-12)

- **Weeks 1-4:** Python advanced patterns, Git, Docker setup. Math deep dive (Linear Algebra).
- **Weeks 5-8:** Calculus & Physics. **Deliverable:** Differentiable Double Pendulum in JAX.
- **Weeks 9-12:** Probability & LLM basics. **Deliverable:** Telemetry-GPT (NanoGPT on race data).

Phase 1: Engineering Stack (Weeks 13-20)

- **Weeks 13-16:** JAX/Flax Deep Dive. **Deliverable:** JAX-ResNet with Hydra/WandB integration.
- **Weeks 17-20:** Building the F1TENTH Gym wrapper in JAX. **Deliverable:** A JAX environment where a random agent can step through the sim.

Phase 2: World Models (Weeks 21-32)

- **Weeks 21-24:** VAE & RSSM implementation. **Deliverable:** A trained VAE that can reconstruct LiDAR heatmaps.
- **Weeks 25-28:** Dynamics Model training. **Deliverable:** A model that can "dream" a valid trajectory of the car 50 steps into the future.
- **Weeks 29-32:** Agent (Actor-Critic) integration. **Deliverable:** An agent learning to drive in the "Dream".

Phase 3: Robustness & Sim2Real (Weeks 33-44)

- **Weeks 33-36:** Domain Randomization implementation.
- **Weeks 37-40:** MPPI Planner implementation (Model-Based Control).
- **Weeks 41-44:** Competitive benchmarks (Racing vs. Baseline PPO).

Phase 4: Deployment & Portfolio (Weeks 45-56+)

- **Weeks 45-48:** ROS 2 Bridge & Hardware Safety Layer.
- **Weeks 49-52:** Real-world testing (or high-fidelity Sim testing if hardware unavailable).
- **Weeks 53-56:** Final Polish. Video production. Portfolio website construction. Mock Interviews.

---

10. Conclusion

This plan is not merely a checklist; it is a curriculum for intellectual transformation. By adhering to this roadmap, you are not just building a racing car; you are building a **differentiable, model-based artificial intelligence capable of reasoning about physical dynamics**. This is the precise skill set sought by elite labs like DeepMind, OpenAI, and calibration-heavy autonomous driving companies.
The path requires discipline. The "Foundations" phase will feel slow. The "Debugging JAX" phase will be frustrating. But the result—a verifiable, high-performance, first-principles AI system—is the gold standard of modern research engineering. Proceed with rigor.

Detailed Technical Appendix: The Science of NeuroSim

This section provides the granular "how-to" scientific details referenced in the strategic plan, bridging the gap between high-level strategy and keyboard-level implementation.

A. The Math of World Models

The core mathematical object we are learning is the joint distribution of the sequence of observations $o_{1:T}$ and actions $a_{1:T}$.
We maximize the Evidence Lower Bound (ELBO):

\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(z_{1:T}|o_{1:T}, a_{1:T})} \left

- **Reconstruction:** Can the model generate the sensor data from its hidden state?
- **KL Divergence:** Does the "posterior" (state inferred from seeing the image) match the

"prior" (state predicted just from history)? Minimizing this term trains the agent to *predict the future*.

B. Differentiable Physics Implementation

To make the physics engine differentiable, we cannot use standard if/else logic for collisions, as this breaks the gradient.

- **Soft Contacts:** Instead of hard constraints, we use penalty functions (spring-damper models) to model collisions. $F_{contact} = -k \cdot \text{penetration} - b \cdot \text{velocity}$.
- **Smooth Approximations:** Replace max(0, x) (ReLU) with jax.nn.softplus(x) to ensure gradients flow smoothly near zero.

C. The "DeepMind" Interview Prep

DeepMind interviews differ from standard SWE interviews. They focus on:

1. **Linear Algebra intuition:** "What happens to the eigenvalues of a matrix if I add a row of zeros?"
2. **ML Theory:** "Derive the update rule for Backpropagation Through Time (BPTT)." "Why does VAE collapse occur?"
3. **Research Coding:** "Implement K-Means clustering in JAX using vmap." (They look for vectorization skills, not for loops).

This master plan provides the artifacts to answer these questions not just with theory, but with "I built a system that uses this." That is the winning differentiator.

Works cited

1. How I Prepared for DeepMind and Google AI Research Internship Interviews in 2019, accessed on February 5, 2026, https://davidstutz.de/how-i-prepared-for-deepmind-and-google-ai-research-internship-interviews-in-2019/
2. Physics of Learning: A Lagrangian perspective to different learning paradigms - arXiv, accessed on February 5, 2026, https://arxiv.org/html/2509.21049v1
3. Hamiltonian mechanics - Wikipedia, accessed on February 5, 2026, https://en.wikipedia.org/wiki/Hamiltonian_mechanics
4. Lagrangian Neural Networks - Natural Intelligence, accessed on February 5, 2026, https://greydanus.github.io/2020/03/10/lagrangian-nns/
5. Simplifying Hamiltonian and Lagrangian Neural Networks via Explicit Constraints - NeurIPS, accessed on February 5, 2026, https://proceedings.neurips.cc/paper/2020/file/9f655cc8884fda7ad6d8a6fb15cc001e-Paper.pdf
6. AI discovers PHYSICS: Lagrange & Hamiltonian (MIT) - YouTube, accessed on February 5, 2026, https://www.youtube.com/watch?v=wrkiMZ3SKH4
7. Model-Based Reinforcement Learning:Theory and Practice - Berkeley AI Research, accessed on February 5, 2026, https://bair.berkeley.edu/blog/2019/12/12/mbpo/

8. World-Models 🌍 Model Based Reinforcement Learning - YouTube, accessed on February 5, 2026, https://www.youtube.com/watch?v=0MjI2NA_s4c
9. jax-ml/jax: Composable transformations of Python+NumPy programs: differentiate, vectorize, JIT to GPU/TPU, and more - GitHub, accessed on February 5, 2026, https://github.com/jax-ml/jax
10. Using JAX to accelerate our research - Google DeepMind, accessed on February 5, 2026, https://deepmind.google/blog/using-jax-to-accelerate-our-research/
11. Writing a Training Loop in JAX and Flax - Wandb, accessed on February 5, 2026, https://wandb.ai/jax-series/simple-training-loop/reports/Writing-a-Training-Loop-in-JAX-and-Flax--VmlldzoyMzA4ODEy
12. Tutorial 2 (JAX): Introduction to JAX+Flax — UvA DL Notebooks v1.2 documentation, accessed on February 5, 2026, https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/JAX/tutorial2/Introduction_to_JAX.html
13. Configuring Experiments with Hydra - xlstm-jax documentation - Read the Docs, accessed on February 5, 2026, https://xlstm-jax.readthedocs.io/en/latest/configuration_with_hydra.html
14. Machine Learning Articles, Tutorials & Reports by Weights & Biases - Wandb, accessed on February 5, 2026, https://wandb.ai/fully-connected/projects/Hydra
15. fratambot/MLOps-with-wandb: A keras project with Weights & Biases integration to achieve complete MLOps maturity for experimentation and model development - GitHub, accessed on February 5, 2026, https://github.com/fratambot/MLOps-with-wandb
16. Best Practices for Structuring and Naming Artifactory Repositories | JFrog, accessed on February 5, 2026, https://jfrog.com/whitepaper/best-practices-structuring-naming-artifactory-repositories/
17. [D] How do you structure and organize your ML/DL project code? - Reddit, accessed on February 5, 2026, https://www.reddit.com/r/MachineLearning/comments/g8h58c/d_how_do_you_structure_and_organize_your_mldl/
18. TransDreamerV3: Implanting Transformer In DreamerV3 - arXiv, accessed on February 5, 2026, https://arxiv.org/html/2506.17103v1
19. danijar/dreamerv3: Mastering Diverse Domains through World Models - GitHub, accessed on February 5, 2026, https://github.com/danijar/dreamerv3
20. The Delusional Driver: Reimplementing World Models in JAX | by Sergey Tiraspolsky, accessed on February 5, 2026, https://medium.com/@sergeyt/world-models-in-jax-solving-the-sim2real-gap-293222d709cc
21. Learning from Simulation, Racing in Reality - ETH Zurich Research Collection, accessed on February 5, 2026, https://www.research-collection.ethz.ch/bitstreams/f9cf3476-e856-43dc-97e8-c35d8ec65361/download
22. f1tenth.dev - An Open-source ROS based F1/10 Autonomous Racing Simulator - Madhur Behl, accessed on February 5, 2026,

https://www.madhurbehl.com/newpubs/varundevsureshbabumadhurbehl2020.pdf
23. F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning, accessed on February 5, 2026, http://proceedings.mlr.press/v123/o-kelly20a/o-kelly20a.pdf
24. UNDERSTANDING DOMAIN RANDOMIZATION FOR SIM-TO-REAL TRANSFER - OpenReview, accessed on February 5, 2026, https://openreview.net/pdf?id=T8vZHIRTrY
25. Sim-to-Real Policy and Reward Transfer with Adaptive Forward Dynamics Model - IEEE Xplore, accessed on February 5, 2026, https://ieeexplore.ieee.org/iel7/10160211/10160212/10161298.pdf
26. f1tenth/f1tenth_baselines: Baseline solutions to the three benchmarks. - GitHub, accessed on February 5, 2026, https://github.com/f1tenth/f1tenth_baselines
27. How to get started with Reinforcement Learning (RL) | by Aleksa Gordić - Medium, accessed on February 5, 2026, https://gordicaleksa.medium.com/how-to-get-started-with-reinforcement-learning-rl-4922fafeaf8c
28. [2011.05617] Sim-To-Real Transfer for Miniature Autonomous Car Racing - arXiv, accessed on February 5, 2026, https://arxiv.org/abs/2011.05617
29. Learning-Based Lap Time Optimization Using F1TENTH Simulation : 8 Steps - Instructables, accessed on February 5, 2026, https://www.instructables.com/Learning-Based-Lap-Time-Optimization-Using-F1TENTH/
30. Advancing Autonomous Racing: A Comprehensive Survey of the RoboRacer (F1TENTH) Platform - arXiv, accessed on February 5, 2026, https://www.arxiv.org/pdf/2506.15899
31. Best Coding Practices to Ensure Reproducibility - Westat, accessed on February 5, 2026, https://www.westat.com/wp-content/uploads/legacy/RiveroWestatIssueBrief.pdf